ENTERPRISE DIGITAL BANKING SYSTEM

07

# Deployment and Operations

Running locally, Docker and Docker Compose, environment configuration and profiles, CI/CD pipeline, logging, monitoring, backup and recovery, troubleshooting.

| | |
|---|---|
| Document | 07 — Deployment and Operations |
| System | Enterprise Digital Banking System (banking-app) + notification-service |
| Stack | Java 17 · Spring Boot 4.1 · Spring Security + JWT · Spring Data JPA · MySQL 8 · Apache Kafka 3.8 · Redis 7 · Docker |
| Repository | Enterprise-Digital-Banking-System (branch `master`) |
| Version / date | 1.0 — 13 October 2026 |
| Author | hemantwint-hey |

Status tags used throughout: **BUILT** exists in the code today **PLANNED** part of the target design (roadmap docs 01–09), not yet implemented **GAP** known limitation

## Contents


1. 1. Environments
2. 2. Running Locally (today)
3. 3. Docker PLANNED
4. 4. Configuration
5. 5. CI/CD Pipeline PLANNED
6. 6. AWS Deployment PLANNED
7. 7. Logging
8. 8. Monitoring PLANNED
9. 9. Backup and Recovery
10. 10. Troubleshooting Runbook

# 1. Environments

| Environment | How it runs | Profile | Schema | Status |
|---|---|---|---|---|
| Local (IDE) | `./mvnw spring-boot:run` ; MySQL local; Kafka from notification-service compose file | default / `dev` | `ddl-auto=update` | BUILT |
| Local (containers) | `docker compose up -d` — full stack | `prod` | Flyway + `validate` | PLANNED |
| CI | GitHub Actions: build + tests | `test` | H2 / Testcontainers | PLANNED |
| Cloud | AWS Elastic Beanstalk + RDS MySQL | `prod` | Flyway + `validate` | PLANNED |

## 2. Running Locally (today)

### 2.1 Prerequisites

- JDK 17+, Git, Docker Desktop (for Kafka), MySQL 8 running on `localhost:3306`.
- Sibling repository `notification-service` (optional, for notifications).

### 2.2 Steps

```
# 1. Kafka (single-node KRaft) - compose file lives in notification-service
cd ../notification-service && docker compose up -d && docker ps      # notification-kafka is Up

# 2. Environment (PowerShell)
$env:DB_PASSWORD="your-mysql-password"
$env:JWT_SECRET="at-least-32-random-bytes-base64-encoded-secret-value"

# 3. banking-app (port 8080) - creates banking_db automatically
cd ../Enterprise-Digital-Banking-System
./mvnw spring-boot:run

# 4. notification-service (port 8081) - in another terminal
cd ../notification-service && ./mvnw spring-boot:run

# 5. Smoke test
curl -X POST http://localhost:8080/auth/register -H "Content-Type: application/json" -d '{...}'
```

# 3. Docker PLANNED

## 3.1 Dockerfile (multi-stage)

```
# --- build stage ---
FROM eclipse-temurin:17-jdk AS build
WORKDIR /app
COPY .mvn .mvn
COPY mvnw pom.xml ./
RUN ./mvnw -q dependency:go-offline          # cached layer: dependencies
COPY src src
RUN ./mvnw -q clean package -DskipTests

# --- run stage ---
FROM eclipse-temurin:17-jre
WORKDIR /app
RUN useradd -r -u 1001 bank                  # do not run as root
COPY --from=build /app/target/*.jar app.jar
USER bank
EXPOSE 8080
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-jar", "app.jar"]
```

```
docker build -t banking-app .
docker run -p 8080:8080 -e DB_PASSWORD=... -e JWT_SECRET=... banking-app
```

### 3.2 docker-compose.yml — full stack

```
services:
  mysql:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: ${DB_PASSWORD}
      MYSQL_DATABASE: banking_db
    volumes: [ "mysql-data:/var/lib/mysql" ]
    healthcheck: { test: ["CMD", "mysqladmin", "ping", "-h", "localhost"], interval: 10s, retries: 10 }

  redis:
    image: redis:7

  kafka:
    image: apache/kafka:3.8.0
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_LISTENERS: PLAINTEXT://0.0.0.0:9092,CONTROLLER://0.0.0.0:9093
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

  banking-app:
    build: .
    depends_on: { mysql: { condition: service_healthy }, redis: { condition: service_started }, kafka: { condition:
service_started } }
    expose: [ "8080" ]
    environment:
      SPRING_PROFILES_ACTIVE: prod
      DB_PASSWORD: ${DB_PASSWORD}
      JWT_SECRET: ${JWT_SECRET}
      SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/banking_db
      SPRING_DATA_REDIS_HOST: redis
      SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:9092

  notification-service:
    build: ../notification-service
    depends_on: [ mysql, kafka ]
    expose: [ "8081" ]
    environment:
      DB_PASSWORD: ${DB_PASSWORD}
      SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/notification_db?createDatabaseIfNotExist=true
      SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:9092
      MAIL_USERNAME: ${MAIL_USERNAME}
      MAIL_PASSWORD: ${MAIL_PASSWORD}

  nginx:
    image: nginx:1.27-alpine
    ports: [ "80:80" ]
    volumes: [ "./nginx/nginx.conf:/etc/nginx/nginx.conf:ro" ]
    depends_on: [ banking-app, notification-service ]

  prometheus:
    image: prom/prometheus
    volumes: [ "./monitoring/prometheus.yml:/etc/prometheus/prometheus.yml:ro" ]
  grafana:
    image: grafana/grafana
    ports: [ "3000:3000" ]

volumes:
  mysql-data:
```

> Inside Compose, services reach each other by **service name** ( `kafka:9092` , `mysql:3306` ), never `localhost` . Only Nginx (and Grafana for the operator) publish ports to the host.

```
docker compose up -d --build     # start everything
docker compose ps                 # all services Up / healthy
docker compose logs -f banking-app
docker compose down               # stop (add -v to also delete the MySQL volume)
```

# 4. Configuration

## 4.1 Properties reference

| Property | Env override | Default | Notes |
|---|---|---|---|
| `spring.datasource.url` | SPRING_DATASOURCE_URL | jdbc:mysql://localhost:3306/banking_db?createDatabaseIfNotExist=true | |
| `spring.datasource.username` | DB_USERNAME | root | |
| `spring.datasource.password` | DB_PASSWORD | — | required |
| `spring.jpa.hibernate.ddl-auto` | — | update | `validate` in prod |
| `spring.jpa.show-sql` | — | true | false in prod |
| `jwt.secret` | JWT_SECRET | — | required, ≥ 32 bytes |
| `jwt.expiration` | JWT_EXPIRATION | 86400000 | ms |
| `spring.kafka.bootstrap-servers` | KAFKA_BOOTSTRAP_SERVERS | localhost:9092 | |
| `spring.data.redis.host/port` | SPRING_DATA_REDIS_HOST | localhost / 6379 | PLANNED |
| `spring.cache.redis.time-to-live` | — | 600000 | 10 min PLANNED |
| `management.endpoints.web.exposure.include` | — | health,info,metrics,prometheus | PLANNED |
| `server.forward-headers-strategy` | — | native (prod) | trust Nginx X-Forwarded-* PLANNED |
| `spring.datasource.hikari.maximum-pool-size` | — | 10 | tune under load |

## 4.2 Profiles PLANNED

```
# application-dev.properties
spring.jpa.show-sql=true
logging.level.org.example.enterprisedigitalbankingsystem=DEBUG

# application-prod.properties
spring.jpa.show-sql=false
spring.jpa.hibernate.ddl-auto=validate
spring.flyway.enabled=true
server.forward-headers-strategy=native
logging.level.org.example.enterprisedigitalbankingsystem=INFO
```

Activate with `--spring.profiles.active=dev` or `SPRING_PROFILES_ACTIVE=prod`.

## 5. CI/CD Pipeline PLANNED

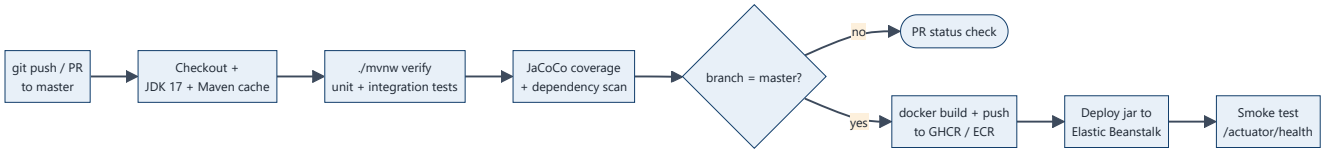



Figure 1 — CI/CD pipeline

```yaml
# .github/workflows/ci.yml
name: CI
on:
  push: { branches: [ master ] }
  pull_request:
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: '17', cache: maven }
      - run: ./mvnw -B verify
        env:
          JWT_SECRET: ${{ secrets.TEST_JWT_SECRET }}
      - uses: actions/upload-artifact@v4
        if: always()
        with: { name: test-reports, path: target/surefire-reports }
  deploy:
    needs: build
    if: github.ref == 'refs/heads/master'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: '17', cache: maven }
      - run: ./mvnw -B -DskipTests package
      - uses: einaregilsson/beanstalk-deploy@v22
        with:
          aws_access_key: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws_secret_key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          application_name: banking-app
          environment_name: banking-app-prod
          region: ap-south-1
          version_label: ${{ github.sha }}
          deployment_package: target/Enterprise-Digital-Banking-System-0.0.1-SNAPSHOT.jar
```

## 6. AWS Deployment PLANNED

1. Build: `./mvnw clean package -DskipTests` → `target/Enterprise-Digital-Banking-System-0.0.1-SNAPSHOT.jar`.
2. Create an **RDS MySQL 8** instance (private subnet, security group allowing 3306 only from the EB instances).
3. Create an **Elastic Beanstalk** Java (Corretto 17) application; upload the jar (console or `eb init` / `eb create`).
4. Configuration → Software → Environment properties: `SPRING_PROFILES_ACTIVE=prod`, `DB_PASSWORD`, `JWT_SECRET`, `SPRING_DATASOURCE_URL=jdbc:mysql://<rds-endpoint>:3306/banking_db`, `SERVER_PORT=5000` (EB's default proxy port).
5. Attach an ACM certificate to the load balancer for HTTPS.
6. Kafka (MSK) and Redis (ElastiCache) are deferred: they stay containerised and documented. Without a broker, producers log errors and transfers still succeed.
7. Verify: `curl https://<eb-url>/actuator/health` → `{"status":"UP"}`.

# 7. Logging

## 7.1 Configuration PLANNED

```
<!-- src/main/resources/logback-spring.xml -->
<configuration>
  <include resource="org/springframework/boot/logging/logback/defaults.xml"/>
  <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
    <encoder><pattern>%d{HH:mm:ss.SSS} %-5level [%thread] %X{traceId:-} %logger{36} - %msg%n</pattern></encoder>
  </appender>
  <logger name="org.example.enterprisedigitalbankingsystem" level="INFO"/>
  <logger name="org.hibernate.SQL" level="DEBUG"/>
  <logger name="org.springframework" level="WARN"/>
  <root level="INFO"><appender-ref ref="CONSOLE"/></root>
</configuration>
```

## 7.2 Logging rules

| Level | Use for |
|---|---|
| ERROR | Unexpected failures needing action (Kafka publish failed, unhandled exception) |
| WARN | Recoverable problems (rate limiter unavailable → fail-open, retries) |
| INFO | Business milestones (transfer completed with reference, loan approved) — no PII |
| DEBUG | Diagnostics, SQL (dev only) |

Never log passwords, JWTs, full Aadhaar/PAN, or request bodies of auth endpoints. Log the transaction *reference*, not the account holder's details. Logs are distinct from the audit trail: logs are for operators and may be rotated; `audit_logs` is a compliance record.

## 8. Monitoring PLANNED

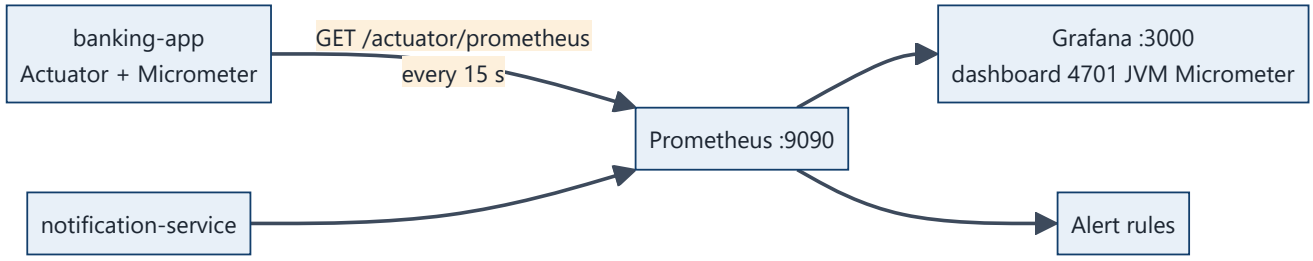



Figure 2 — Metrics pipeline

```
<dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<dependency><groupId>io.micrometer</groupId><artifactId>micrometer-registry-prometheus</artifactId></dependency>

management.endpoints.web.exposure.include=health,info,metrics,prometheus
management.endpoint.health.show-details=always

# monitoring/prometheus.yml
scrape_configs:
  - job_name: banking-app
    metrics_path: /actuator/prometheus
    static_configs: [ { targets: ['banking-app:8080'] } ]
```

| Signal | Metric | Alert when |
|---|---|---|
| Availability | `up{job="banking-app"}` , `/actuator/health` | down > 1 min |
| Error rate | `http_server_requests_seconds_count{status=~"5.."}` | > 2 % of requests for 5 min |
| Latency | `http_server_requests_seconds` p95 for `/transactions/transfer` | > 500 ms |
| JVM | `jvm_memory_used_bytes` , GC pause | heap > 85 % |
| DB pool | `hikaricp_connections_pending` | > 0 sustained |
| Kafka | consumer lag ( `kafka-consumer-groups.sh --describe --group notification-service` ) | lag growing for 10 min |
| Business | custom counters: transfers, failed logins (Micrometer `Counter` ) | failed logins spike |

# 9. Backup and Recovery

| Item | Approach |
|---|---|
| Objectives | RPO ≤ 15 min (max data loss), RTO ≤ 1 h (max downtime) |
| Local / Compose | Nightly logical dump: `docker exec mysql mysqldump -uroot -p$DB_PASSWORD --single-transaction --routines banking_db \| gzip > backup/banking_$(date +%F).sql.gz` ; keep 14 days |
| AWS RDS | Automated backups (7–35 days) + point-in-time recovery from binlogs; manual snapshot before every release |
| Schema | Recreated from Flyway migrations in git |
| Kafka | Not a system of record. Events lost before consumption only cost a notification; source of truth is MySQL |
| Redis | Cache only — no backup needed; flushed on restore |
| Secrets | Stored in EB / Secrets Manager, not in backups |

## 9.1 Restore procedure

1. Stop banking-app (prevent writes).
2. Restore: `gunzip -c banking_YYYY-MM-DD.sql.gz \| mysql -uroot -p banking_db` (RDS: restore snapshot / PITR to a new instance and switch the URL).
3. Start the app; Flyway validates the schema version.
4. Flush Redis ( `redis-cli FLUSHALL` ) so no pre-restore balances are served.
5. Reconcile: for every account, `balance` must equal the last `ledger_entries.balance_after_entry` .
6. Test the restore quarterly — an untested backup is not a backup.

```
-- Reconciliation query
SELECT a.id, a.account_number, a.balance, le.balance_after_entry
FROM accounts a
JOIN ledger_entries le ON le.id = (SELECT MAX(id) FROM ledger_entries WHERE account_id = a.id)
WHERE a.balance <> le.balance_after_entry;    -- must return 0 rows
```

## 10. Troubleshooting Runbook

| Symptom | Likely cause | Fix |
|---|---|---|
| App fails at startup: `Could not resolve placeholder 'jwt.secret'` | `JWT_SECRET` not set | Set env var in run config / compose |
| `WeakKeyException` | JWT secret shorter than 256 bits | Use `openssl rand -base64 48` |
| `Access denied for user 'root'` | Wrong `DB_PASSWORD` | Fix env var |
| `Communications link failure` | MySQL not running / wrong host (localhost vs mysql in compose) | Start MySQL; check datasource URL |
| Every request returns 401 | Missing `Bearer` prefix, expired token, or secret changed since token was issued | Log in again; check header |
| Transfers succeed but no notification | Kafka down, topic name typo (e.g. stray space), notification-service not running, SMTP creds unset (status FAILED) | `docker ps`; check both apps' bootstrap servers; check consumer logs |
| Producer logs `Failed to publish ...Event` | Broker unreachable | Start Kafka; events are not retried — affected notifications are lost |
| Intermittent 500 on concurrent deposits | `ObjectOptimisticLockingFailureException` | Expected protection; client retries (409 mapping planned) |
| Stale balance returned | Missing `@CacheEvict` on a write path | Add eviction; `redis-cli FLUSHALL` |
| Flyway `Validate failed: Migration checksum mismatch` | An applied migration file was edited | Revert the edit; create a new V<n> file |
| Customer delete returns 500 | FK: customer still has accounts / loans / beneficiaries | Close/remove children first |